

Lab 03: Split Routes and Reuse a Service Function

Maps to: A3, A8

Objective: Refactor a monolithic FastAPI file into router and service modules without changing the contract.

Build: main.py, routes.py and service.py

Tools: FastAPI APIRouter, pytest, OpenAI Python SDK

Lab asset inventory

- ai_vibe.py - OpenAI Python SDK runner with Pydantic structured output and safe generated-file paths
- mock-data.json - authorised synthetic scenario, route contract and test cases
- mock-database.sqlite - inspectable SQLite database seeded only with synthetic commerce and CRM records
- Prompts.pdf - learner-facing first-run, refinement and review prompts
- Prompts.md - copyable source used by the SDK runner
- working-files/ - complete FastAPI, SQLite, HTML/CSS/JavaScript project, including VS Code REST Client requests

AI vibe-coding control loop

Prompts.pdf → ai_vibe.py → Responses API → CodeBundle → working-files/generated/ → pytest + human review

Detailed procedure

Step 1 - Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

1. Perform the action exactly as stated: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 2 - Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

1. Perform the action exactly as stated: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 3 - Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

1. Perform the action exactly as stated: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 4 - Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

1. Perform the action exactly as stated: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 5 - Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

1. Perform the action exactly as stated: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 6 - Review the generated diff and copy only accepted changes into the working application.

1. Perform the action exactly as stated: Review the generated diff and copy only accepted changes into the working application.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 7 - Run the baseline tests and record the passing count.

1. Perform the action exactly as stated: Run the baseline tests and record the passing count.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 8 - Move product decision logic into service.py.

1. Perform the action exactly as stated: Move product decision logic into service.py.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 9 - Create an APIRouter with prefix /api/v1/products and tag products.

1. Perform the action exactly as stated: Create an APIRouter with prefix /api/v1/products and tag products.
2. Observe the resulting file, HTTP response, log or browser state.

3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 10 - Include the router once in main.py.

1. Perform the action exactly as stated: Include the router once in main.py.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 11 - Run the original tests and compare /openapi.json before and after.

1. Perform the action exactly as stated: Run the original tests and compare /openapi.json before and after.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 12 - Remove duplicate imports and document module ownership in README.md.

1. Perform the action exactly as stated: Remove duplicate imports and document module ownership in README.md.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 13 - Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

1. Perform the action exactly as stated: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 14 - Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

1. Perform the action exactly as stated: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Acceptance criteria

- All baseline tests still pass and OpenAPI exposes each route exactly once.
- Evidence names the lab number and the mapped codes: A3, A8.
- The saved SDK evidence records the prompt, model, assumptions, generated file list and verification result.

- mock-database.sqlite contains synthetic seed data only; no .env, live credential, virtual environment or runtime northstar.db is submitted.

Evidence checklist

- ☐ Prompts.pdf reviewed and dry run captured
- ☐ mock-data.json contains synthetic data only
- ☐ mock-database.sqlite opens in VS Code SQLite Viewer
- ☐ SDK CodeBundle saved under working-files/generated/
- ☐ Generated diff reviewed before applying
- ☐ Working artifact
- ☐ Request/response or test output
- ☐ One failure diagnosis
- ☐ Acceptance criterion confirmed